Sudoku Solver GenAI Comparison

1. Objective

Make the sudoku solver put each solution in a file.

GenAI Access and Shared Prompt

How I accessed the programs: I used Gemini through its web interface and CoPilot through its desktop application

Prompt used for both GenAIs:

> Write the solutions for each puzzle file to a file named solutionN.txt, where N is the puzzle number. Each solutionN.txt file should include the following: o If no solutions are found, a single line: “No solution found.” o If solutions are found, it should include all solutions followed by a single line: “Solutions found: n”, where n is the number of solutions found. o The solutions should be in the same format as the input files: puzzle1.txt – puzzle5.txt.

```
 Program: EECS 348 Assignment 4 - Sudoku Solver
 Description: Read five 9x9 Sudoku puzzles and find every solution using
              an object-oriented recursive depth-first backtracking search.
 Author: Shimrah
 Created: October 14, 2026
 Collaborators: None
 Sources: ChatGPT (https://chatgpt.com/) and Google Gemini
          (https://gemini.google.com/)
*/

#include <iostream>
#include <fstream>
#include <string>
using namespace std;

// Based on ChatGPT's original class, with input and search improvements.
class SudokuSolver {
private:
    int grid[9][9]{};    // Zero means the square is empty.
    int solutionCount = 0; // Count every solution found for this puzzle.

    // Check whether a number can be placed at the requested position.
    bool allowed(int row, int col, int value) const {
        // Check the entire row and column for a duplicate number.
        for (int i = 0; i < 9; ++i) {
            if (grid[row][i] == value || grid[i][col] == value)
                return false;
        }
        // Locate the upper-left corner of the matching 3x3 box.
        int boxRow = (row / 3) * 3;
```

```
        int boxCol = (col / 3) * 3;
        // Check every square in that box for the same number.
        for (int r = boxRow; r < boxRow + 3; ++r) {
            for (int c = boxCol; c < boxCol + 3; ++c) {
                if (grid[r][c] == value)
                    return false;
            }
        }
        return true; // Number does not conflict with Sudoku rules.
    }

    // Improved search: pick the empty square with the fewest candidates.
    bool findBestEmpty(int& bestRow, int& bestCol) const {
        int fewest = 10; // At most nine numbers could fit a square.
        bool found = false; // Track whether any empty square exists.
        // Examine every square to find the most restricted empty one.
        for (int row = 0; row < 9; ++row) {
            for (int col = 0; col < 9; ++col) {
                if (grid[row][col] != 0)
                    continue; // Do not change a number from the puzzle.
                found = true;
                int choices = 0; // Count numbers allowed in this square.
                for (int value = 1; value <= 9; ++value) {
                    if (allowed(row, col, value))
                        ++choices;
                }
                // Remember the square with the smallest candidate count.
                if (choices < fewest) {
                       fewest = choices;
                    bestRow = row;
                    bestCol = col;
                }
                if (fewest == 0)
                    return true; // This branch cannot lead to a solution.
            }
        }
        return found; // False means that the entire grid is filled.
    }

    // Print the current Sudoku grid, with underscores for blank squares.
    void printGrid() const {
        for (int row = 0; row < 9; ++row) {
            for (int col = 0; col < 9; ++col) {
                char shown = grid[row][col] == 0 ? '_' : char('0' + grid[row]
[col]);
                cout << shown << (col == 8 ? '\n' : ' ');
            }
        }
    }

    // Based on ChatGPT's recursive method; uses improved cell selection.
    void solveRecursive() {
        int row = -1, col = -1; // Position selected for the next guess.
        if (!findBestEmpty(row, col)) { // No blanks: a solution is complete.
```

```
            ++solutionCount;
            cout << "Solution " << solutionCount << ":\n";
            printGrid(); // Print immediately instead of storing solutions.
            return;
        }
        // Try all possible values for this empty position.
        for (int value = 1; value <= 9; ++value) {
            if (allowed(row, col, value)) {
                grid[row][col] = value; // Make a guess.
                solveRecursive();       // Continue solving this guess.
                grid[row][col] = 0;     // Undo it to explore other guesses.
            }
        }
    }

public:
    // Load exactly 81 squares; reject invalid characters or extra values.
    bool load(const string& fileName) {
        ifstream file(fileName); // Open the named puzzle file.
        if (!file)
            return false; // File could not be opened.
        for (int row = 0; row < 9; ++row) {
            for (int col = 0; col < 9; ++col) {
                char ch;
                if (!(file >> ch) || (ch != '_' && (ch < '1' || ch > '9')))
                    return false; // Missing or illegal character.
                grid[row][col] = ch == '_' ? 0 : ch - '0';
            }
        }
        char extra;
        if (file >> extra)
            return false; // Reject any data after the 81st square.
        // Temporarily remove each clue to detect conflicting clues.
        for (int row = 0; row < 9; ++row) {
            for (int col = 0; col < 9; ++col) {
                if (grid[row][col] == 0)
                    continue;
                int value = grid[row][col];
                grid[row][col] = 0; // Exclude this clue from its own test.
                bool valid = allowed(row, col, value);
                grid[row][col] = value; // Restore the original clue.
                if (!valid)
                    return false; // The clues contradict Sudoku rules.
            }
        }
        solutionCount = 0; // Reset the count for a new file.
        return true; // File contains a well-formed, conflict-free puzzle.
    }

    // Handle one puzzle from file loading through solving and printing.
    void run(const string& fileName) {
        cout << "File: " << fileName << "\n";
        if (!load(fileName)) {
            cout << "Could not read a valid puzzle.\n\n";
```

```
            return;
        }
        cout << "Original puzzle:\n";
        printGrid(); // Show the puzzle before filling any squares.
        solveRecursive(); // Search every valid branch recursively.
        if (solutionCount == 0)
            cout << "No solution found\n";
        cout << "\n";
    }
};

// Process the five input files using separate solver objects.
int main() {
    for (int i = 1; i <= 5; ++i) {
        SudokuSolver solver; // Fresh puzzle and solution counter each time.
        solver.run("puzzle" + to_string(i) + ".txt");
    }
    return 0; // All five files have been processed.
}
```

> here's the format:

> 5_____17_1_65_9_4_4721_6___9_____5__18_954__6__4_2389_4___93__9_7_3_5_263958714

---

3. GenAI #1

GenAI Used: Gemini

Code

```
#include <fstream>
#include <iostream>
#include <string>
using namespace std;

class SudokuSolver {
private:
    int grid[9][9]{};
    int solutionCount = 0;

    bool allowed(int row, int col, int value) const {
        for (int i = 0; i < 9; ++i) {
            if (grid[row][i] == value || grid[i][col] == value)
                return false;
        }

        int boxRow = (row / 3) * 3;
```

```
        int boxCol = (col / 3) * 3;
        for (int r = boxRow; r < boxRow + 3; ++r) {
            for (int c = boxCol; c < boxCol + 3; ++c) {
                if (grid[r][c] == value)
                    return false;
            }
        }
        return true;
    }

    bool findBestEmpty(int& bestRow, int& bestCol) const {
        int fewest = 10;
        bool found = false;
        for (int row = 0; row < 9; ++row) {
            for (int col = 0; col < 9; ++col) {
                if (grid[row][col] != 0)
                    continue;

                found = true;
                int choices = 0;
                for (int value = 1; value <= 9; ++value) {
                    if (allowed(row, col, value))
                        ++choices;
                }
                if (choices < fewest) {
                    fewest = choices;
                    bestRow = row;
                    bestCol = col;
                }
                if (fewest == 0)
                    return true;
            }
        }
        return found;
    }

    void writeGrid(ostream& out) const {
        for (int row = 0; row < 9; ++row) {
            for (int col = 0; col < 9; ++col) {
                char value = grid[row][col] == 0
                    ? '_'
                    : static_cast<char>('0' + grid[row][col]);
                out << value << (col == 8 ? '\n' : ' ');
            }
        }
    }

    void solveRecursive(ostream& out) {
        int row = -1;
        int col = -1;
        if (!findBestEmpty(row, col)) {
            writeGrid(out);
            out << '\n';
            ++solutionCount;
```

```
            return;
        }

        for (int value = 1; value <= 9; ++value) {
            if (allowed(row, col, value)) {
                grid[row][col] = value;
                solveRecursive(out);
                grid[row][col] = 0;
            }
        }
    }

    bool load(const string& fileName) {
        ifstream file(fileName);
        if (!file)
            return false;

        for (int row = 0; row < 9; ++row) {
            for (int col = 0; col < 9; ++col) {
                char value;
                if (!(file >> value) ||
                    (value != '_' && (value < '1' || value > '9'))) {
                    return false;
                }
                grid[row][col] = value == '_' ? 0 : value - '0';
            }
        }

        char extra;
        if (file >> extra)
            return false;

        for (int row = 0; row < 9; ++row) {
            for (int col = 0; col < 9; ++col) {
                if (grid[row][col] == 0)
                    continue;

                int value = grid[row][col];
                grid[row][col] = 0;
                bool valid = allowed(row, col, value);
                grid[row][col] = value;
                if (!valid)
                    return false;
            }
        }

        solutionCount = 0;
        return true;
    }

public:
    bool run(const string& inputFileName, const string& outputFileName) {
        ofstream outFile(outputFileName);
        if (!outFile) {
```

```
            cerr << "Could not open output file: " << outputFileName << '\n';
            return false;
        }

        if (!load(inputFileName)) {
            outFile << "No solution found.\n";
        } else {
            solveRecursive(outFile);
            if (solutionCount == 0) {
                outFile << "No solution found.\n";
            } else {
                outFile << "Solutions found: " << solutionCount << '\n';
            }
        }

        if (!outFile) {
            cerr << "Could not write output file: " << outputFileName << '\n';
            return false;
        }
        return true;
    }
};

int main() {
    bool successful = true;
    for (int i = 1; i <= 5; ++i) {
        SudokuSolver solver;
        string inputFile = "puzzle" + to_string(i) + ".txt";
        string outputFile = "solution" + to_string(i) + ".txt";
        if (!solver.run(inputFile, outputFile))
            successful = false;
    }
    return successful ? 0 : 1;
}
```

4. GenAI #2

GenAI Used: CoPilot

Code

```
#include <fstream>
#include <iostream>
#include <string>
using namespace std;

class SudokuSolver {
private:
```

```
    int grid[9][9]{};
    int solutionCount = 0;

    bool allowed(int row, int col, int value) const {
        for (int i = 0; i < 9; ++i) {
            if (grid[row][i] == value || grid[i][col] == value)
                return false;
        }

        int boxRow = (row / 3) * 3;
        int boxCol = (col / 3) * 3;
        for (int r = boxRow; r < boxRow + 3; ++r) {
            for (int c = boxCol; c < boxCol + 3; ++c) {
                if (grid[r][c] == value)
                    return false;
            }
        }
        return true;
    }

    bool findBestEmpty(int& bestRow, int& bestCol) const {
        int fewest = 10;
        bool found = false;
        for (int row = 0; row < 9; ++row) {
            for (int col = 0; col < 9; ++col) {
                if (grid[row][col] != 0)
                    continue;

                found = true;
                int choices = 0;
                for (int value = 1; value <= 9; ++value) {
                    if (allowed(row, col, value))
                        ++choices;
                }
                if (choices < fewest) {
                    fewest = choices;
                    bestRow = row;
                    bestCol = col;
                }
                if (fewest == 0)
                    return true;
            }
        }
        return found;
    }

    void writeGrid(ostream& out) const {
        for (int row = 0; row < 9; ++row) {
            for (int col = 0; col < 9; ++col) {
                char value = grid[row][col] == 0
                    ? '_'
                    : static_cast<char>('0' + grid[row][col]);
                out << value << (col == 8 ? '\n' : ' ');
            }
```

```
        }
    }

    void solveRecursive(ostream& out) {
        int row = -1;
        int col = -1;
        if (!findBestEmpty(row, col)) {
            writeGrid(out);
            out << '\n';
            ++solutionCount;
            return;
        }

        for (int value = 1; value <= 9; ++value) {
            if (allowed(row, col, value)) {
                grid[row][col] = value;
                solveRecursive(out);
                grid[row][col] = 0;
            }
        }
    }

    bool load(const string& fileName) {
        ifstream file(fileName);
        if (!file)
            return false;

        for (int row = 0; row < 9; ++row) {
            for (int col = 0; col < 9; ++col) {
                char value;
                if (!(file >> value) ||
                    (value != '_' && (value < '1' || value > '9'))) {
                    return false;
                }
                grid[row][col] = value == '_' ? 0 : value - '0';
            }
        }

        char extra;
        if (file >> extra)
            return false;

        for (int row = 0; row < 9; ++row) {
            for (int col = 0; col < 9; ++col) {
                if (grid[row][col] == 0)
                    continue;

                int value = grid[row][col];
                grid[row][col] = 0;
                bool valid = allowed(row, col, value);
                grid[row][col] = value;
                if (!valid)
                    return false;
            }
```

```
        }

        solutionCount = 0;
        return true;
    }

public:
    bool run(const string& inputFileName, const string& outputFileName) {
        ofstream outFile(outputFileName);
        if (!outFile) {
            cerr << "Could not open output file: " << outputFileName << '\n';
            return false;
        }

        if (!load(inputFileName)) {
            outFile << "No solution found.\n";
        } else {
            solveRecursive(outFile);
            if (solutionCount == 0) {
                outFile << "No solution found.\n";
            } else {
                outFile << "Solutions found: " << solutionCount << '\n';
            }
        }

        if (!outFile) {
            cerr << "Could not write output file: " << outputFileName << '\n';
            return false;
        }
        return true;
    }
};

int main() {
    bool successful = true;
    for (int i = 1; i <= 5; ++i) {
        SudokuSolver solver;
        string inputFile = "puzzle" + to_string(i) + ".txt";
        string outputFile = "solution" + to_string(i) + ".txt";
        if (!solver.run(inputFile, outputFile))
            successful = false;
    }
    return successful ? 0 : 1;
}
```

Correctness

- both programs check the input tokens and reject duplicate clues, and search for all correct solutions
- both programs correctly use the file i/o to create the solution files
- The biggest issue is error handling. both of the programs return incorrect input as 'no solution found' which is not always the case

Execution Time

- Both programs use brute-force backtracking, so the worst-case running time is exponential in the number of empty cells: roughly O(9^k) in the worst case where k cells are blank.
- Both programs come from the same starter code, so they follow the same algorithm and have no meaningful speed difference

Space Complexity

- both programs keep one 9x9 grid and stream solutions directly to the file, so the depth is at most the number of initial empty cells because of recursion

Maintainability

- gemini uses more comments to explain the algorithm, but its `run()` does not report file open or write failures
- The copilot implememtation has clearer error-reporting and returns failure for output errors. It also uses exit codes in its `main()` which makes the code easier to debug and read.

5. Comparison and Selection

- Because these programs are so similar, I am selecting the Gemini simply because the comments make it easier to follow

6. Improvements Identified for the Selected Program

- make sure invalid input is caught
- add seprate input errors for debugging instead of just 'no solution found'